

CQRS(Command Query Responsibility Segregation)

0.The Core Problem CQRS Solves

In traditional applications:

- One model
- One service
- One repository

is used for everything.

This leads to:

- Mixed read & write logic
- Bloated services
- Hard-to-test code
- Tight coupling
- Difficult scaling

CQRS fixes this by explicitly separating responsibilities.

1. What is CQRS?

CQRS (Command Query Responsibility Segregation) is an architectural pattern that **separates** read operations from write operations.

Command → Changes system state (Create, Update, Delete)

Query → Reads data (Get)

Instead of one model handling both reads and writes, CQRS uses **separate models and handlers**.

2. Traditional CRUD vs CQRS (Quick Comparison)

Traditional CRUD

CQRS

Same model for read & write Separate models

Simple to start

Scales better

Traditional CRUD

Business logic mixed

Hard to optimize

CQRS

Clear responsibility

Read & write optimized separately

3. Why Use CQRS?

CQRS is **not mandatory**, but useful when:

1. Application has **complex business rules**
2. Read operations are **much heavier** than writes
3. Need **high scalability**
4. Need **clear separation of concerns**
5. Want **clean architecture + testability**

When NOT to use CQRS

- Small CRUD apps
 - Simple admin panels
-

4. Core CQRS Concepts

4.1 Command

- Represents an **intention to change data**
 - “I want to do something”
- Example:
- CreateProductCommand
- UpdateProductCommand
- DeleteProductCommand

4.2 Command Handler

- Executes business logic for the command
- Executes **exactly one use case**
- Coordinates:
 - Validation

- Domain rules
- Persistence
- Has **one responsibility**
- Example: CreateProductCommandHandler

4.3 Query

- Used to **fetch data only**
- A **Query** represents a **request for data only**. (Give me Information)
- Example: GetProductsQuery

Why Separate from Commands?

Because:

- Reads are **much more frequent**
- Reads need **different optimization**
- Reads should never change data

4.4 Query Handler

- Executes read logic
- Fetches data
- Performs NO state changes
- Returns DTOs (not entities)

4.5 CQRS Rules Enforced by Design

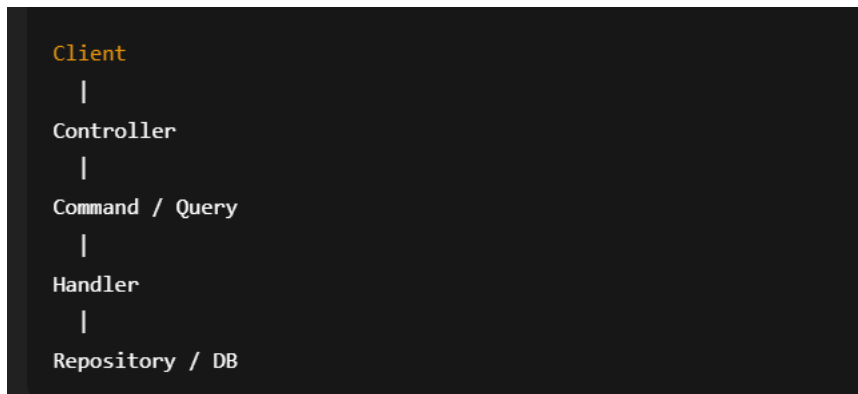
Rule	Enforced By
Commands change state	Command handlers
Queries read data	Query handlers
No data return from commands	Command design
No state change in queries	Query design
Clear intent	Explicit naming

4.6 Real-World Analogy (Very Effective)

Think of a bank:

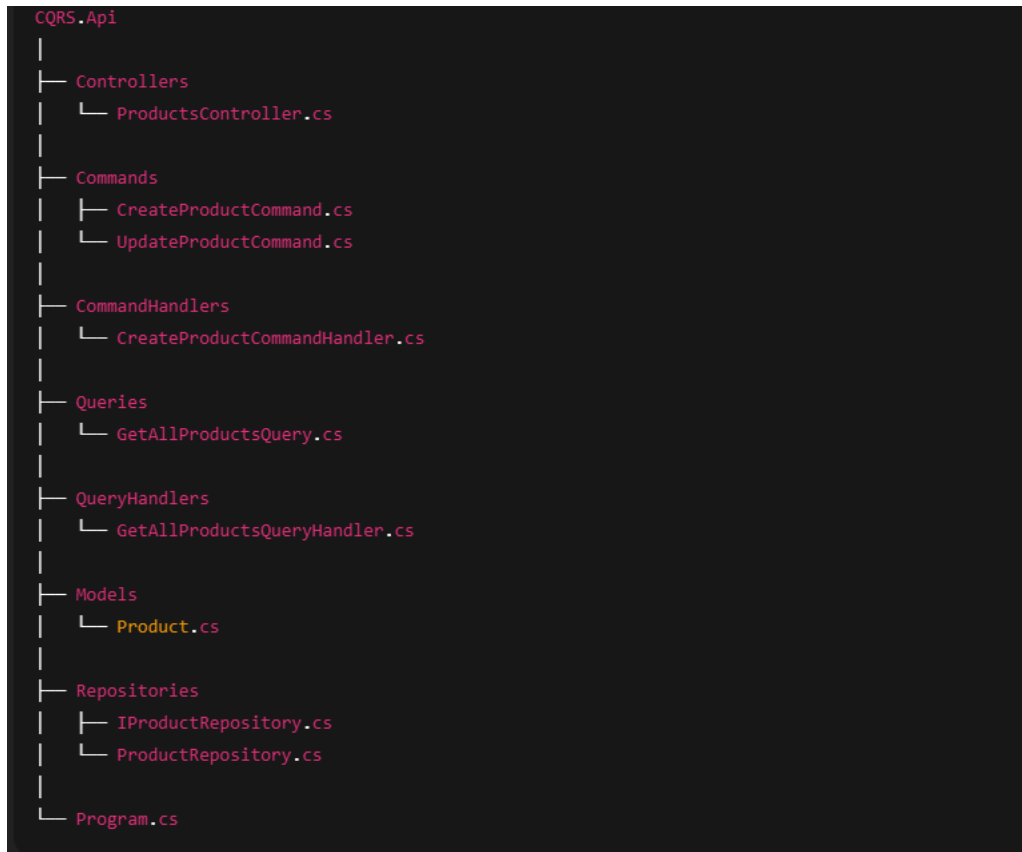
Action	CQRS Component
Deposit money	Command
Deposit processing	Command Handler
Check balance	Query
Balance retrieval	Query Handler

5. CQRS Flow in REST API



6. Building ASP.NET Core REST API Using CQRS (Simple Version)

7. Project Structure (Recommended)



8. Step-by-Step Implementation

8.1 Entity (Domain Model)

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
}
```

8.2 Command (Write Model)

```
public class CreateProductCommand
{
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
```

```
}
```

8.3 Command Handler

```
public class CreateProductCommandHandler
{
    private readonly IProductRepository _repository;

    public CreateProductCommandHandler(IProductRepository repository)
    {
        _repository = repository;
    }

    public void Handle(CreateProductCommand command)
    {
        var product = new Product
        {
            Name = command.Name,
            Price = command.Price
        };

        _repository.Add(product);
    }
}
```

8.4 Query (Read Model)

```
public class GetAllProductsQuery
{
}
```

8.5 Query Handler

```
public class GetAllProductsQueryHandler
{
    private readonly IProductRepository _repository;

    public GetAllProductsQueryHandler(IProductRepository repository)
    {
        _repository = repository;
    }

    public IEnumerable<Product> Handle(GetAllProductsQuery query)
    {
        return _repository.GetAll();
    }
}
```

8.6 Repository

```
public interface IProductRepository
{
    void Add(Product product);
    IEnumerable<Product> GetAll();
}

public class ProductRepository : IProductRepository
{
    private static readonly List<Product> _products = new();

    public void Add(Product product)
```

```

{
    product.Id = _products.Count + 1;
    _products.Add(product);
}

public IEnumerable<Product> GetAll()
{
    return _products;
}
}

```

8.7 Controller (Very Important in CQRS)

```

[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{
    private readonly CreateProductCommandHandler _createHandler;
    private readonly GetAllProductsQueryHandler _getHandler;

    public ProductsController(
        CreateProductCommandHandler createHandler,
        GetAllProductsQueryHandler getHandler)
    {
        _createHandler = createHandler;
        _getHandler = getHandler;
    }

    [HttpPost]

```



```
public IActionResult Create(CreateProductCommand command)
{
    _createHandler.Handle(command);
    return Ok("Product Created");
}

[HttpGet]
public IActionResult GetAll()
{
    var result = _getHandler.Handle(new GetAllProductsQuery());
    return Ok(result);
}
}
```

8.8 Dependency Injection (Program.cs)

```
builder.Services.AddSingleton<IProductRepository, ProductRepository>();
```

```
builder.Services.AddTransient<CreateProductCommandHandler>();
```

```
builder.Services.AddTransient<GetAllProductsQueryHandler>();
```

9. Key CQRS Rules to Remember

1. **Commands never return data**
2. **Queries never modify data**
3. **Use DTOs for queries**
4. **Controllers are thin**
5. **Handlers contain business logic**

10.CQRS Rules (Very Important)

Rule	Applied
Commands modify state	✓
Queries read data	✓
Commands return nothing	✓
Queries return DTOs	✓
Async end-to-end	✓
Domain independent	✓